

The backdrop layer - a background the host draws...

The backdrop layer: a background the host draws beneath RISC OS

How the RISC OS Pi 4 emulator came to draw the desktop's background itself, at the window's own resolution, beneath RISC OS's windows, icons and menus. RISC OS decides which pixels are background by setting two bits in each pixel's top byte, and no RISC OS code was changed to do it.

The Acorn theme used to paint its sage ground and ghost-acorn watermark inside RISC OS, at the resolution of the screen mode. The host then magnified the whole desktop to fit the window, so the acorn was only as sharp as an 800×600 mode. The backdrop layer moves that background to the host. RISC OS still draws every window, icon, menu and the icon bar. It also marks the pixels that belong to the background, and the host draws its own scene through them at full resolution: the acorn as a signed-distance field, a moving version with drifting lights, a tile, or a picture.

It was built on the evening of 14 September 2026: first for the Mac's Metal display and the Mac release, then for the Windows Direct3D 11 display. The Windows release followed early on the 15th. There is no RISC OS module: the guest's part is one sprite file and two changed lines in the boot sequence.

This document describes the fork's `riscos-pi4` branch as published at `d535505fad`, 15 September 2026, and names commits by hash. The [graphics and sound walkthrough](#) covers the decode and scale passes this layer builds on. The [HostFS walkthrough](#) covers the disc folder that the release scripts patch.

These documents

Chapter	What it covers
1. A layer beneath the desktop	the watermark it replaces, the pieces, and the order they landed in
2. One byte, two bits	the transfer-byte tag, the key colour, and why both are needed
3. Painting the tag from the guest	a tagged sprite, the pinboard, the Wimp flag, and the release patches
4. Holes, then over	the decode pass makes holes, and the scale pass composites over the scene
5. The acorn as a distance	the watermark rebuilt from its curves, and the moving scene

These documents, continued

Chapter	What it covers
6. Tiles and pictures	images from ImageIO and WIC, tile sizing, and filling the window
7. The gate and the menu	off by default, the Backdrop menu on both hosts, and the hint
8. Capturing what is on screen	screenshots and QMP screendump of the composite, and the test harness
9. Measured, and in progress	what the layer costs, and work in progress

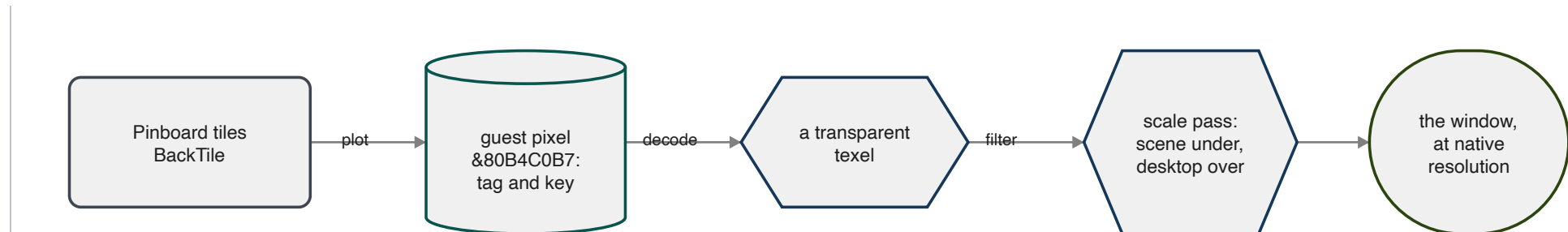
At a glance

The tag	bits 7–6 of a 32bpp pixel's transfer byte: 10 below, 01 above (reserved), 00/11 nothing
The key	a below pixel must also be the Acorn sage #B7C0B4, so that an EOR drag box stays visible
Guest side	a 256×256 tagged sprite tiled by Pinboard, plus one Wimp flag; no code changes
Host side	the decode pass returns a transparent texel; the scale pass composites $c.rgb + bg \cdot (1 - c.a)$

At a glance, continued

Scenes	acorn, acorn-live, tile:<file>, picture:<file>, none; off (the default) removes the feature
Cost	0.156 ms for RISC OS to redraw the whole background; 0.02–0.14 ms of GPU time per frame on the Mac
Code	ui/metal.m, ui/dx11.cpp, ui/console.c, riscos-pi4/tools/mkbacktile.py, the release scripts

One pixel, from the pinboard to the window



Reading the diagrams

Every diagram colours its outlines by what each box is:

Outline	Meaning
dark blue	code in this fork: the display front ends, the tools, the scripts
grey	code from elsewhere: QEMU, RISC OS, the host operating system
purple	a file or data artefact: a sprite, an image, a setting
teal	state at run time: guest memory, a texture, a buffer
amber	a decision or a test
green	the outcome the user sees

1. A layer beneath the desktop

This chapter sets the scene. It describes the background that RISC OS used to draw for itself and why that stopped being good enough. It then states the idea in one paragraph, lists the constraints the design had to meet, shows where each piece lives, and gives the order in which the commits landed.

The watermark it replaces

On 13 September the fork's desktop design settled on an Acorn look: a flat sage ground, dark ink for pinboard text, and a quiet watermark in the middle. The watermark is a "ghost" acorn, the acorn silhouette in a single tone just off the ground (#ADB7A8 on #B7C0B4), so it reads as a mark rather than a picture. A small design tool made it as a 220-pixel, 32bpp sprite. The boot sequence's `PinSetup` file then asked Pinboard to centre that sprite on a sage fill:

```
Backdrop -Centre Boot:Resources.!ThemeDefs.Themes.Acorn.Backdrop -Colour &B4C0B700
```

RISC OS drew the result into its framebuffer at the resolution of the screen mode. In the 800×600 desktop mode the icon bar takes the bottom 66 rows, which leaves an 800×534 background. The emulator's display front end then magnified the whole desktop to fit the window. A 1646×1156 window on a Retina Mac is about twice the mode's size in each direction, so the acorn's edge was only as crisp as the scaler could make an 800×600 image. On a 4K screen it was softer still.

A picture backdrop had a second problem: it was expensive for RISC OS to draw. Measured on an M4 Max with the end-user disc, one full redraw of the 800×534 background cost 0.505 ms with a picture held as a cached sprite, and 48.3 ms with a JPEG. Pinboard redraws parts of the background whenever a window moves over it.

The idea

The host should draw the background itself, at the window's own resolution and beneath the desktop. RISC OS should say which of its pixels are background. The host then composites the desktop over its own scene, so windows, menus, icons and the icon bar stay RISC OS's pixels, magnified exactly as before. Only the background pixels are replaced, and the replacement can be any size, any sharpness, and even animated, at no cost to the guest.

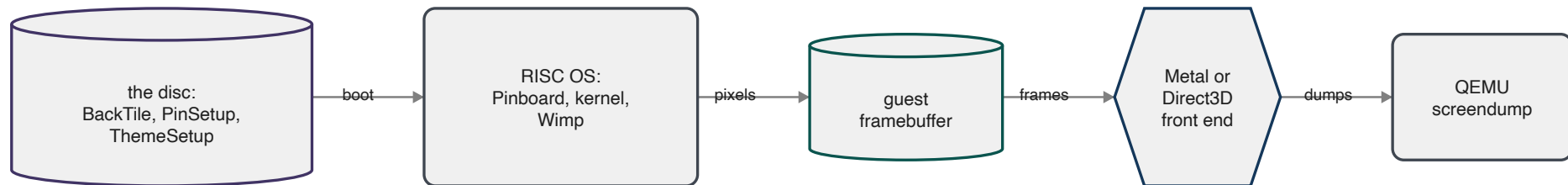
The constraints it had to meet

Four constraints shaped every later decision:

Constraint	How the design meets it
No RISC OS code changes	Pinboard tiles a tagged sprite; the kernel's plot and the Wimp's copies keep the byte
Nothing changes when it is off	backdrop=off is the default: the byte is ignored and no menu is built
The same picture on both hosts	one key colour, one set of constants, the acorn maths ported line for line to HLSL
A disc that looks right anywhere	the tile is plain sage, so any other display shows a flat Acorn ground

The last row matters for users. The disc that ships with a release can be booted by an older emulator, by QEMU's own `cocoa` display, or with the layer switched off. In each case, the tile's own colour is what appears.

Where the pieces live



The files

Piece	Files	Chapter
the tagged tile	riscos-pi4/tools/mkbacktile.py	3
the disc patches	riscos-pi4/tools/make-release.sh, riscos-pi4/tools/make-release.py	3
the launchers	riscos-pi4/app/launcher.zsh (Mac), riscos-pi4/app/win/launcher.c (Windows)	7
the Mac front end	ui/metal.m, ui/metal.c, ui/metal.h	4 to 8

The files, continued

Piece	Files	Chapter
the Windows front end	ui/dx11.cpp, ui/dx11.c, ui/dx11.h, meson.build	4 to 8
the display option	qapi/ui.json: backdrop on DisplayMetal and DisplayDx11	7
the screendump hook	include/ui/console.h, ui/console.c, ui/ui-qmp-cmds.c	8
the test harness	riscos-pi4/tools/run.py	8

The design notes are in `riscos-pi4/MACOS.md`, section 7a, "The backdrop layer", and in `riscos-pi4/tools/README.md`.

The order it landed in

Seven commits over ten hours:

Commit	When	What it added
9a9cd13460	14 Sep, 18:36	Metal: the layer, the tag, both acorn scenes, pictures, a pipeline fix
42ee6cf20a	14 Sep, 18:36	the Mac release: mkbacktile.py, BACKDROP=, the launcher
1cc6d04a18	14 Sep, 18:44	Metal: the Backdrop menu, tile:, the 4096 cap, MacOS.md §7a
e80479db19	14 Sep, 20:08	the acorn's early exit, the 256-pixel tile, the measurements

The order it landed in, continued

Commit	When	What it added
84dfd3c071	14 Sep, 20:20	the gate: off, the default, removes the feature and its menu
c4024d236f	14 Sep, 21:47	Direct3D 11: the layer and its menu; screen capture of the composite; run.py
d535505fad	15 Sep, 04:46	the Windows release: the tile on the disc, backdrop=acorn in the launcher

The first two commits share a timestamp to the second: they are the feature and its release packaging. Further work on the Mac menu was in progress but not yet committed when this document was written. Chapter 9 describes it.

What this is not

The layer is not a RISC OS module, and it is not an emulated device. The guest runs no new code. It uses a byte that every 32bpp pixel already has and that the operating system leaves at zero. All the new code is in the two display front ends that already decode the guest's framebuffer on the GPU. The layer is one more decision in their shaders and one more texture in their scale pass.

This has a practical consequence. The layer can only show through what RISC OS draws in a 32bpp mode, because only 32bpp pixels have a spare byte. In a 16-bit or 8-bit mode there is no tag, the desktop shows no holes, and the Backdrop menu explains why (chapter 7).

2. One byte, two bits

The whole contract between RISC OS and the host is one convention about one byte. This chapter describes the byte, the two bits that carry the tag, the colour test added on top of them, and how the host reads both from a pixel. The same convention is written out in four places: `MACOS.md §7a`, the header comment of `mkbacktile.py`, and comments in `ui/metal.m` and `ui/dx11.cpp`.

A 32bpp pixel has a spare byte

In a 32bpp RISC OS screen mode, each pixel is one little-endian word written as `&TTBBGGRR`. The low three bytes are red, green and blue. The top byte is the **transfer** byte, which RISC OS also calls the supremacy byte:

Byte in memory	Bits of the word	Holds
+0	7–0	red
+1	15–8	green
+2	23–16	blue
+3	31–24	the transfer byte: the tag

Everything the operating system draws leaves the transfer byte at zero, because the kernel clears it when it plots. The claim was checked before anything was built on it. A read of the live framebuffer of the Acorn desktop, taken out of guest memory, found the byte at zero on all 480,000 pixels. The byte was therefore free to carry information that RISC OS would never disturb by accident.

Bits 7 and 6 are the tag

Only the top two bits carry meaning. The other six are reserved:

Bits 7–6	Meaning
10	below: the layer beneath shows through this pixel
01	above: reserved for a layer drawn over this pixel, such as an overlay
00	nothing: an ordinary pixel, which is what RISC OS writes
11	nothing: this is what the &FF written by some sprite tools looks like

Two choices in that table are deliberate.

- **&FF means nothing.** Some sprite tools write `&FF` into the top byte, treating it as an opaque alpha channel. If one bit alone meant "below", sprites from those tools would punch holes wherever they were plotted. Requiring the pattern `10` keeps both the all-zero and the all-ones byte inert.
- **Bits 5–0 are reserved.** The host tests only bits 7–6, masking with `0xC0`, so a later use of the low six bits will not change how an older host reads the tag. Writers should set them to zero, as `mkbacktile.py` does.

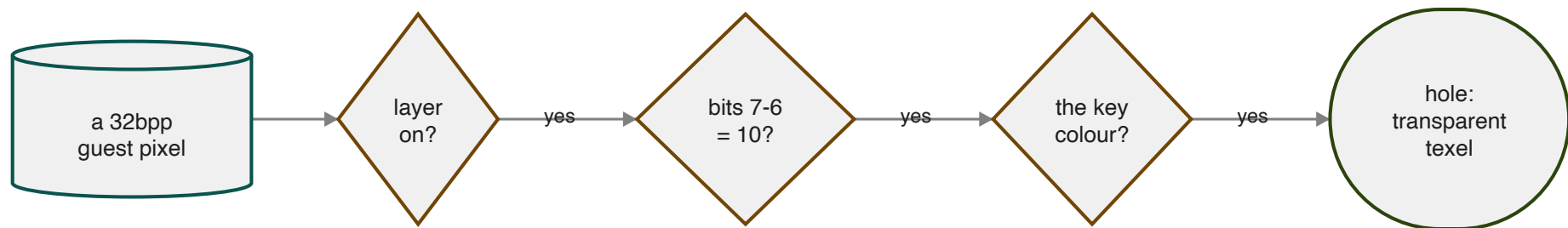
A tag alone is not enough: the key colour

A pixel tagged "below" becomes a hole only if it is also the backdrop **key colour**, the Acorn theme's sage `#B7C0B4`. The reason is the Wimp's exclusive-OR drawing.

When you drag a window outline or a rubber band across the desktop, the Wimp draws the box by EOR-ing a colour into the pixels underneath and EOR-ing it again to remove it. Over a tagged background, those box pixels change colour. Whether the EOR also touches the transfer byte depends on how the operation is done. If the tag survived and the host tested only the tag, the drag box would turn into holes, and you would drag an invisible outline across the picture.

Testing the colour as well settles the question. Any change to the pixel's colour takes it out of the key, so the host shows the pixel as ordinary desktop, and the box stays visible over the scene. When the EOR is undone, the pixel returns to sage with its tag, and the hole reappears. The commit that introduced the layer, `9a9cd13460`, records that windows, menus, a dragged window and a rubber band over the backdrop were all checked on screen.

The host's test, as a decision



Every "no" exits to the same place: the pixel's own colour, fully opaque, as the decode pass has always returned it. The first test is not on the pixel at all. It checks whether the layer is switched on and has a scene, so a machine without the layer never looks at the byte (chapter 7).

The tagged pixel as a number

`mkbacktile.py` builds the word from the key colour and the tag:

Part	Value	Shifted into place
red	0xB7	0x000000B7
green	0xC0	0x0000C000
blue	0xB4	0x00B40000
tag "below", reserved bits zero	0x80	0x80000000
the word		0x80B4C0B7 , stored as bytes B7 C0 B4 80

The host front ends compare the colour without the tag, so their key constant is `0x00B4C0B7`, in `0x00BBGGRR` order. It is the same number in both front ends:

```
#define METAL_BACKDROP_KEY    0x00B4C0B7u    /* 0x00BBGGRR: #B7C0B4 */
#define DX11_BACKDROP_KEY    0x00B4C0B7u    /* 0x00BBGGRR: #B7C0B4 */
```

The key is passed to the shaders at run time, in the first word of the backdrop constants, rather than compiled into the shader source. The pinboard's own fill colour is the same sage, written for `*Backdrop` as `&B4C0B700`, which is RISC OS's `&BBGGRR00` colour form.

Reading it in the shader

The Metal decode pass reads the four bytes of the pixel from the guest framebuffer directly. A pixel-order flag swaps red and blue for framebuffers that store BGR. The shader source is held in `ui/metal.m` as a C string, and this excerpt, like the others in this document, leaves out the string quoting:

```
if (BPP == 32) {
    uint tag;
    r = raw[row + bx * 4 + 0];
    g = raw[row + bx * 4 + 1];
    b = raw[row + bx * 4 + 2];
    tag = raw[row + bx * 4 + 3];
    if (P.misc.z == 0) { uint t = r; r = b; b = t; } /* BGR order */
    /* The transfer byte's layer tag, bits 7-6: 10 is below, and a
     * below pixel in the backdrop key colour is a hole the layer
     * beneath shows through -- transparent, premultiplied. */
    if (P.bd.w != 0 && (tag & 0xC0) == 0x80
        && (r | (g << 8) | (b << 16)) == P.bd.x) {
        return float4(0.0);
    }
}
```

The Direct3D decode pass reads a whole word with a helper and shifts the same fields out of it. Its test is the same, character for character, except that the tag comes from `(w >> 24) & 0xC0` and the hole is written `float4(0, 0, 0, 0)`.

What `P.bd` carries

Both shaders receive a four-word `bd` block of constants on every frame:

Word	Metal	Direct3D 11
<code>bd.x</code>	the key, <code>0x00B4C0B7</code>	the key, <code>0x00B4C0B7</code>
<code>bd.y</code>	milliseconds into the live scene's 30-minute loop	the same
<code>bd.z</code>	the icon bar height, 66 guest pixels	the same
<code>bd.w</code>	1 when a scene is drawn, else 0	the scene number, 0 to 4

The only difference is the last word. On the Mac the scene is compiled into the pipeline as a function constant, so the shader only needs an on/off flag. The Windows shader branches on the scene number at run time. Chapter 4 compares the two approaches. In both, `bd.w` is zero when the layer is off or the scene is `none`, and then the decode pass returns tagged sage pixels as ordinary sage.

Why not another way

Other ways for the host to find the background come to mind. None of them is discussed in the fork's notes; this is a comparison made for this document. Each fails at least one of the constraints from chapter 1, while the tag costs nothing to paint and moves with the pixels whenever the Wimp copies a block of the screen:

Alternative	Where it falls short
treat every sage pixel as background	a sage icon, a sage window or sage text would become a hole
ask Pinboard for the background rectangles	needs guest code and a new channel, and goes stale as soon as a window moves
a second framebuffer holding a mask	RISC OS would have to draw everything twice
read the transfer byte as alpha	RISC OS writes zero everywhere, which as alpha would make the whole desktop transparent

3. Painting the tag from the guest

RISC OS has to put the tag on exactly the pixels that are background, keep it there while windows move, and never put it anywhere else. This chapter shows how the disc does that with one generated sprite and two edited lines. It walks through the tool that writes the sprite, byte by byte, and explains why the tile is 256 pixels. It ends with the release scripts that make the change to the disc a user installs.

No RISC OS code changes

Three facts about RISC OS make the approach possible. MACOS.md §7a records each of them as measured on a running machine, by reading the framebuffer out of guest memory:

- **Pinboard already tiles sprites.** `*Backdrop -Tile` plots a sprite repeatedly across the whole background. The tag only has to be in the sprite.
- **The kernel's sprite plot writes all 32 bits.** A sprite pixel's transfer byte reaches the framebuffer unchanged. The tag therefore lands on exactly the background, and windows, menus, icons and their labels, which RISC OS draws itself, stay untagged.
- **The Wimp's block copies carry the byte with the pixels.** When a window is dragged, the Wimp copies screen blocks and asks Pinboard to redraw the area it uncovers. Pinboard's redraw re-tags that area, and nothing stale is left behind.

Without the layer, the tile is simply the sage ground. The old watermark sat on the same colour, so the disc looks like a plain Acorn desktop on any other display.

Two edited lines

The Acorn theme's boot files change in two places:

File	Before	After
!Boot/Choices/Boot/Tasks/PinSetup	Backdrop -Centre ... Themes.Acorn.Backdrop	Backdrop -Tile ...Themes.Acorn.BackTile
!Boot/Choices/Boot/PreDesk/ThemeSetup	WimpVisualFlags -RemoveIconBoxes	WimpVisualFlags -RemoveIconBoxes - NoIconBoxesInTransWindows

The first line swaps the centred watermark sprite for the tiled, tagged one. The ... stands for `Boot:Resources.!ThemeDefs`, and the rest of the line, including the sage `-Colour`, is unchanged.

The second line is less obvious. By default the Wimp fills a small box behind the name of each icon on the pinboard. That box is drawn by the Wimp, so it is untagged. Over the host's layer it would appear as a solid rectangle behind every icon name on the desktop. The flag `-NoIconBoxesInTransWindows` tells the Wimp not to fill those boxes, so the names are drawn straight onto the tagged tile.

The tool: `mckbacktile.py`

`riscos-pi4/tools/mckbacktile.py` writes the sprite file. It is 68 lines, with no dependencies beyond Python's `struct` module. The core is one function:

```
KEY = (0xB7, 0xC0, 0xB4)      # R, G, B: the Acorn theme's sage
TAG_BELOW = 0x80              # transfer byte: layer tag 10, reserved bits 0

def tile(size):
    r, g, b = KEY
    word = struct.pack('<I', r | (g << 8) | (b << 16) | (TAG_BELOW << 24))
    body = word * (size * size)
    hdr = bytearray(44)
    struct.pack_into('<i', hdr, 0, 44 + len(body))          # to the next sprite
    struct.pack_into('<12s', hdr, 4, b'backtile')
    struct.pack_into('<7i', hdr, 16,
                      size - 1,                          # width in words - 1
                      size - 1,                          # height - 1
                      0, 31,                              # first and last bit used
                      44, 44,                             # image; mask = image: none
                      1 | (90 << 1) | (90 << 14) | (6 << 27)) # 32bpp, 90 dpi
    sprite = bytes(hdr) + body
    return struct.pack('<III', 1, 16, 16 + len(sprite)) + sprite
```

The pixel word is the `0x80B4C0B7` from chapter 2, repeated `size × size` times. Everything else is the standard RISC OS sprite file layout, described below.

The file header

A sprite file is a sprite area without its first word, the area's total size. So the file starts with three words:

Offset	Value	Meaning
+0	1	the number of sprites in the file
+4	16	where the first sprite starts, counted as if the missing size word were there
+8	$16 + \text{len}(\text{sprite})$	the first free word after the last sprite, counted the same way

The sprite header

The sprite itself starts with a 44-byte header, and the pixels follow at once:

Offset	Value written	Meaning
+0	$44 + \text{len}(\text{body})$	the offset to the next sprite, which is the size of this one
+4	backtile	the name, 12 bytes, zero-padded
+16	$\text{size} - 1$	the width in words, minus one; at 32bpp a pixel is a word
+20	$\text{size} - 1$	the height in rows, minus one

The sprite header, continued

Offset	Value written	Meaning
+24, +28	0, 31	the first and last bits used in each row, so every bit is used
+32	44	where the image starts, from the start of the sprite
+36	44	where the mask starts; equal to the image offset means no mask
+40	the mode word	the sprite type and resolution, below

The tile has no mask, and that matters. Any pixel a mask marked transparent would not be plotted, so it would keep whatever the framebuffer held before, untagged.

The mode word

The last header word describes the pixel format instead of naming a numbered screen mode:

Bits	Field	Value
0	set, so the word describes a format rather than a numbered mode	1
1–13	horizontal resolution, in dots per inch	90
14–26	vertical resolution, in dots per inch	90
27–31	the sprite type	6: 32 bits per pixel

Ninety dots per inch in both directions matches the square-pixel desktop modes the emulator uses, so the tile is plotted pixel for pixel rather than scaled.

Why 256 pixels

A 256×256 tile makes a file of $12 + 44 + 256 \times 256 \times 4 = 262,200$ bytes. That looks wasteful for one colour, and the first version of the tool used a 32-pixel tile. Commit `e80479db19` changed the default after measuring both.

Pinboard's tiled plot makes one sprite plot per tile. By simple arithmetic, a 256-pixel tile covers the 800×534 background in about a dozen plots, where a 32-pixel tile needs more than four hundred. GVFill carries each plot to the host, so the difference is small, but it is measurable:

One full background redraw, measured

M4 Max, end-user disc, GVFill on:

Tile	Per full redraw of 800×534
256 pixels (the default)	0.156 ms
32 pixels	0.170 ms

The size of the file does not matter in practice. It is 262 KB of a single repeated word, which deflates to 325 bytes, so the release's disc zip barely grows. Booted through Pinboard, both sizes tagged the same 424,465 background pixels, with none left untagged.

The release does it for you

Both release scripts apply the same change to the disc they ship. The Mac's `make-release.sh` does it with a short Python script inside the shell script:

```
def patch(rel, old, new, done):
    path = f"{disc}/{rel}"
    s = open(path, "rb").read().decode("latin-1")
    if done in s:
        return
    if s.count(old) != 1:
        sys.exit(f"{rel}: expected one '{old}'")
    open(path, "wb").write(s.replace(old, new).encode("latin-1"))
patch("!Boot/Choices/Boot/Tasks/PinSetup,feb",
      "Backdrop -Centre Boot:Resources.!ThemeDefs.Themes.Acorn.Backdrop",
      "Backdrop -Tile Boot:Resources.!ThemeDefs.Themes.Acorn.BackTile",
      "Themes.Acorn.BackTile")
patch("!Boot/Choices/Boot/PreDesk/ThemeSetup,feb",
      "WimpVisualFlags -RemoveIconBoxes",
      "WimpVisualFlags -RemoveIconBoxes -NoIconBoxesInTransWindows",
      "-NoIconBoxesInTransWindows")
```

Before the patch runs, the shell script checks that the disc has an Acorn theme folder, and writes `BackTile,ff9` into it with `mkbacktile.py`. The `,feb` suffix marks an Obey file and `,ff9` a sprite file, as HostFS names them on the host.

Three guards in eight lines

The `patch` function is small, but it refuses to damage a disc it does not recognise:

Guard	What it prevents
return if the done text is already there	patching the same disc twice
exit unless old occurs exactly once	editing a disc whose boot files are not the expected shape
read and write as Latin-1 bytes	changing any byte outside the edit, whatever the file's character set

On Windows, `make-release.py` has the same function under `configure_backdrop(disc, scene)`, added in `d535505fad`. It uses `io.open` with `newline=""`, so that Windows line-ending translation cannot alter the file, and raises `SystemExit`, with its own wording, in the same two cases.

What BACKDROP chooses

Both scripts take the scene as a setting: `BACKDROP=` for the shell script and `--backdrop` for the Python one. It must be one of three values:

Value	The disc	The app
acorn (the default)	tile and Wimp flag added	starts with the acorn scene and the Backdrop menu
acorn-live	tile and Wimp flag added	starts with the moving acorn and the menu
off	unchanged: the centred watermark stays	the feature and its menu are left out

The two scripts hand the value to the app differently. The Mac script stores it in the app's `Info.plist` as `RISCOSBackdrop`, and the launcher reads it at each start. The Windows script compiles it into the launcher executable with `-DBACKDROP=L"acorn"`. Chapter 7 follows both routes. `d535505fad` shipped the Windows version as the `RISCOSQEA72v7` release.

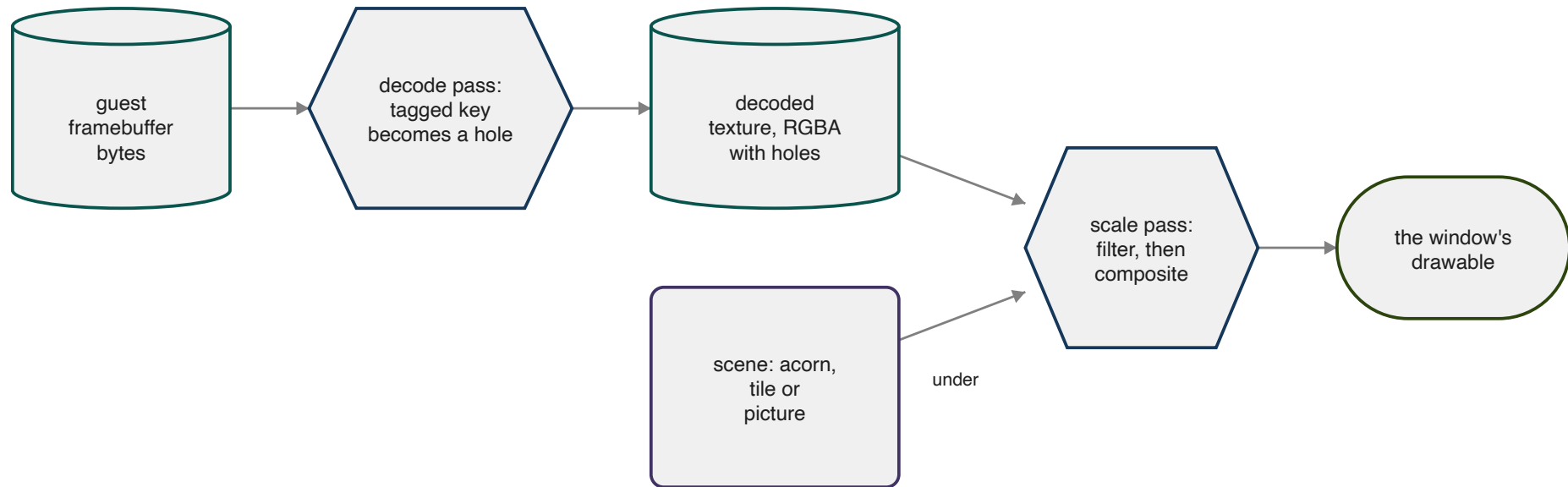
4. Holes, then over

Both display front ends already drew the guest's screen in two GPU passes. The layer adds one decision to the first pass and a scene plus one line of blending to the second. This chapter walks through both passes and explains why the hole is a *premultiplied* transparent texel, with a worked example of an edge pixel. It then compares the Metal and Direct3D versions and ends with a bug in the Mac's scale pipeline that this work uncovered.

Two passes, and what the layer adds

The **decode pass** runs once per guest pixel. It reads the guest's framebuffer bytes, which are raw 32, 24, 16 or palettised 8-bit pixels, and writes a colour texture at the guest's own resolution. The **scale pass** runs once per window pixel. It samples that decoded texture with the chosen filter (sharp bilinear, linear or nearest) and writes the window's drawable. The [graphics walkthrough](#) covers both passes in detail.

The passes with the layer



Nothing else changes. The pointer is still drawn over the scaled frame by its own pass. A guest in a 16-bit mode has no transfer byte and so no holes: every texel stays opaque, and the window looks exactly as it did before.

The hole is premultiplied

The decoded texture stores four channels for every guest pixel. An ordinary pixel is its colour with alpha 1: $(r, g, b, 1)$. A hole is $(0, 0, 0, 0)$, fully transparent. The scale pass then treats every texel as **premultiplied**, meaning its colour channels have already been multiplied by its alpha. It composites the texel over the scene colour bg with the standard "over" formula for premultiplied colour:

$$\text{out.rgb} = \text{c.rgb} + \text{bg} \times (1 - \text{c.a})$$

For an ordinary pixel, c.a is 1, so out is the pixel. For a hole, c is all zeros, so out is the scene. Both of those results would come out the same with straight (non-premultiplied) alpha. The difference is in the pixels between them, because the scale pass filters.

A worked edge

Take a white window border pixel next to a hole, over the acorn scene's sage ground, and a window pixel whose sample lands exactly halfway between the two texels. The bilinear filter averages them. Red channel only, in the range 0 to 1:

Step	Premultiplied, as the shader does it	Straight alpha, for comparison
texels	white $(1, 1)$, hole $(0, 0)$, as (red, alpha)	the same
filtered sample c	red 0.5, alpha 0.5	red 0.5, alpha 0.5
scene bg	sage red $183/255 = 0.718$	0.718
composite	$0.5 + 0.718 \times 0.5 = \mathbf{0.859}$	$0.5 \times 0.5 + 0.718 \times 0.5 = \mathbf{0.609}$
halfway between white and sage	0.859	0.859

The premultiplied result is exactly halfway between white and sage, which is what an edge half-covered by the window should show. The straight-alpha formula comes out darker than both of the colours it blends, because the hole's black colour is counted twice: once by the filter, and again by the alpha weighting. Every window edge over the scene would have a dark fringe. The commit messages describe the chosen behaviour in one phrase: a filtered edge "blends the way every other edge does".

The same arithmetic holds for any number of texels averaged together, which matters on Windows. The Direct3D front end builds a mipmap chain of the decoded texture on every frame and samples it trilinearly, so a window smaller than the mode averages every source pixel it covers. Averaging premultiplied texels gives a correct partial coverage. Averaging straight-alpha texels would not. The Metal front end samples the decoded texture with a linear filter and no mipmaps.

The scale pass, line by line

The end of the Direct3D 11 scale pass, from `ui/dx11.cpp`, after the filtered sample `c` and the optional scanlines. The image branches, elided here, are in chapter 6:

```
if (P.bd.w == 0) {
    return float4(c.rgb, 1);          /* no layer: the guest's pixels */
}

float3 bg;
if (P.bd.w >= 3) {
    ...
} else {
    bg = backdrop_scene(v.pos.xy, outPx, srcPx);
}

/* The decoded texel is premultiplied, so this is "over": where the
 * guest tagged a hole its alpha is 0 and the layer shows whole, and
 * at a filtered edge it shows in proportion. */
return float4(c.rgb + bg * (1.0 - c.a), 1);
```

What each branch does

Scenes are numbered as in the front ends' `enum`. The number is `P.bd.w` on Windows and the `BACKDROP` function constant on the Mac:

Number	Scene	bg comes from
0	off, or none	nothing: the early return gives the guest's pixel, exactly as before the layer
1	acorn	<code>backdrop_scene</code> , the distance field (chapter 5)
2	acorn-live	<code>backdrop_scene</code> , with the lights and vignette (chapter 5)
3	picture:	the image, scaled to fill and centred (chapter 6)
4	tile:	the image, repeated from the top left through a wrapping sampler (chapter 6)

Three details are worth noticing:

- **The early return keeps the old path.** With no layer the pass returns `float4(c.rgb, 1)`, which is what it returned before this work. With no holes in the decoded texture, `c.a` is 1 everywhere, so the composite would give the same answer anyway, but the early return skips the scene entirely.
- **Scanlines never stripe the scene.** The optional CRT scanlines darken `c.rgb` on alternate rows before the composite. At a hole `c.rgb` is zero, so the darkening has nothing to act on, and the scene shows at full brightness. Only the desktop is striped.
- **The output is opaque.** The pass always returns alpha 1, so the window never shows what is behind it.

Metal and Direct3D, side by side

The two front ends compute the same picture from the same numbers, but they were written in their own platform's idiom. The shaders differ in how the scene is selected:

How the scene is selected

	Metal (ui/metal.m)	Direct3D 11 (ui/dx11.cpp)
scene selection	a function constant BACKDROP, fixed when the pipeline is built	a run-time branch on P.bd.w
image texture and sampler	declared only when BACKDROP_IMAGE, that is scene 3 or 4	always bound, as t1 and a wrapping s1; t1 may be empty
changing scene	fb_release_scale() drops the pipeline and the next frame rebuilds it	nothing to rebuild: the next frame's constants carry the scene
decode flag bd.w	1 or 0	the scene number

How the texture is filtered and compiled

	Metal (ui/metal.m)	Direct3D 11 (ui/dx11.cpp)
filtering the decoded texture	linear, no mipmaps	trilinear, over mipmaps generated every frame
shader source	Metal Shading Language, specialised by function constants	HLSL, with #if blocks for pixel format, scaling and scanlines

The practical effect is the same on both hosts: a scene chosen from the menu appears on the next frame the window draws, after a pipeline rebuild on the Mac. 84dfd3c071 records a check on the Mac: choosing Acorn, None, Acorn, None from the menu updated the window within 0.3 s each time, in a resized window too.

When the Mac rebuilds

The Metal side releases the pipeline only when the scene *number* changes. Choosing one tile and then another keeps the same pipeline and swaps only the texture:

```
[backdrop.spec release];
backdrop.spec = [[NSString alloc] initWithUTF8String:spec];
if (mode != backdrop.mode) {
    backdrop.mode = mode;
    fb_release_scale();
}
backdrop.t0 = CFAbsoluteTimeGetCurrent();
```

A fix found on the way

Adding a function constant to the scale pipeline exposed an older bug in the Mac front end. The scale pipeline was built in `metal_backend_init`, before `-display`'s own options had been read, and it was never rebuilt. The `scaling=` and `scanlines=` options therefore never reached the shader. The v1 release binary started with `scanlines=on` shows uniform rows. A build with the fix shows alternating ones.

Since `9a9cd13460`, `fb_release_scale()` drops both the window's scale pipeline and the screenshot pipeline (chapter 8) whenever an option or the scene changes, and the next frame rebuilds them. The comment above `metal_glue_video_opts` puts the cost plainly: "one frame at worst of the old look, which is also what a mode change costs."

5. The acorn as a distance

The `acorn` scene redraws the old watermark on the host, at the window's own resolution, and without a bitmap. The shader describes the acorn by its curves. For every window pixel it computes how far that pixel is from the acorn's edge, then turns the distance into coverage. This chapter walks through the function that does it: where the acorn is placed, the two curves, the step from a curve to a distance, the early exit that made it cheap, and the moving `acorn-live` variant.

Why a distance

A **signed distance function** takes a point and returns how far it is from a shape's edge: positive inside the shape and negative outside. A shader can turn that number into the fraction of a pixel the shape covers. The edge is then antialiased at whatever resolution the window has, and no texture is needed.

The function reproduces the watermark faithfully. At the guest's own resolution, it covers all 37,784 pixels of the 220-pixel sprite RISC OS used to draw, plus 48 sub-pixel extras at the edge. MACOS.md records that a composited desktop differs from the old sprite-drawn one only on the acorn's edge.

Where the acorn sits

The mark is defined in the units of the design's `acorn.svg`, which has a 34-unit view box. The function works about the centre of that box, with `y` increasing downwards. The scene maps those units onto the window:

```
/* view px per guest px, per axis: a window the user has resized away
 * from the mode's shape stretches the desktop unevenly */
float2 k = outPx / srcPx;
/* 220 guest px for the SVG's 34 units, centred above the icon bar,
 * exactly where *Backdrop -Centre put the watermark sprite; the
 * centre follows the stretch, the mark keeps its shape */
const float unitPx = 220.0 / 34.0;
float2 centre = float2(srcPx.x * 0.5, (srcPx.y - float(P.bd.z)) * 0.5) * k;
float d = acorn_distance((px - centre) / (unitPx * k.y));
float cover = clamp(d * unitPx * k.y + 0.5, 0.0, 1.0);
float3 c = lerp(ground, ghost, cover);
```

The centre is the middle of the background area, not the middle of the screen: half the width, and half of the height above the icon bar. It is scaled by `k` on both axes, so it stays centred when the window is stretched. The size uses `k.y` for both axes, so a stretched window moves the acorn without distorting it.

A worked placement

An 800×600 desktop mode in a 1646×1156 Retina window:

Quantity	Expression	Value
stretch k	$1646 / 800, 1156 / 600$	2.058, 1.927
centre, in guest pixels	$800 / 2, (600 - 66) / 2$	400, 267
centre, in window pixels	$\times k$	823, 514.4
one SVG unit	$220 / 34 \times 1.927$	12.47 window pixels
the 34-unit view box	34×12.47	423.9 window pixels tall

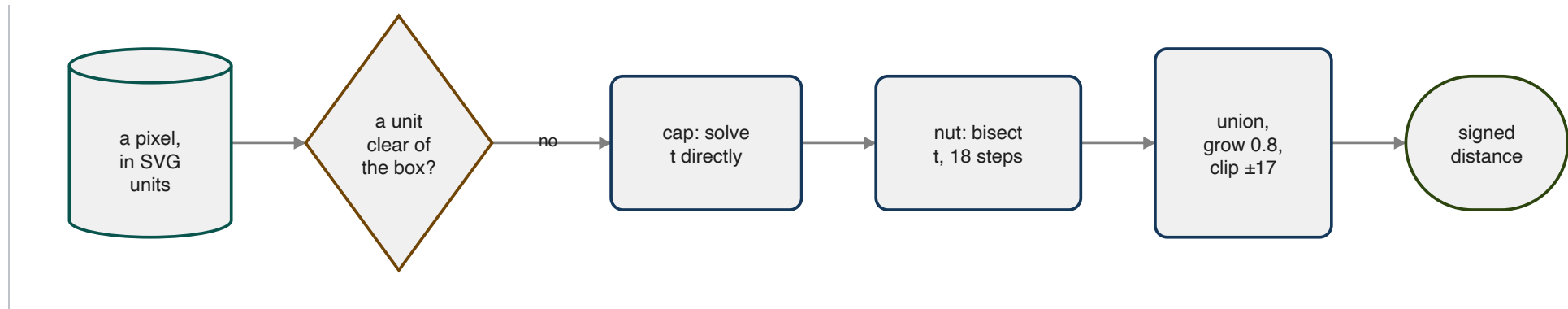
The shape: a cap and a nut

The acorn is the union of two shapes, each bounded by one curve and mirrored about the vertical axis. The function works on the right half only, using $\text{abs}(x)$.

- **The cap** is a quadratic Bézier from $(0, -20)$ with control point $(16, -20)$ to $(15, -7)$. Expanded, it is $x = 32t - 17t^2$ and $y = -20 + 13t^2$. The second equation can be solved for t directly: $t = \sqrt{(y + 20) / 13}$.
- **The nut** is a cubic Bézier from $(0, 18)$ through control points $(9, 18)$ and $(15, 7)$ to $(13, -7)$. Its y falls steadily as t rises, but a cubic has no simple inverse, so the function finds t for a given y by bisection.

The cap's base and the nut's top meet at $y = -7$.

The steps of acorn_distance



A "yes" at the first test returns at once, with the distance to the box as the answer. That exit is what made the scene cheap, and it has its own section below.

From a curve to a distance

For a point at height y , the function finds x_b , the curve's x at that height. The horizontal offset $x_b - |x|$ is positive inside and negative outside. Where the edge slopes, the perpendicular distance is shorter than the horizontal one, so the offset is divided by $\sqrt{1 + (dx/dy)^2}$, using the curve's slope at that height. The result is exact for a straight edge and very close near a gently curved one. Close to the edge is the only place where the value affects coverage.

The first half: the exit and the cap

Here is the first half of the function, in HLSL. The Metal version does the same arithmetic in the same order; only a few names and comments differ:

```
static float acorn_distance(float2 q)
{
    const float grow = 0.8;
    float ax = abs(q.x);

    /* Nothing of the mark reaches past 16 units across or 17 down, so a
     * pixel a unit clear of that box is plainly outside: say so, and skip
     * the curve solve below, which is most of the scene's cost. */
    float outside = max(ax - 16.0, abs(q.y) - 17.0);
    if (outside > 1.0) {
        return -outside;
    }

    /* cap: right side (0,-20) Q (16,-20) (15,-7): x = 32t - 17t^2, y = -20 + 13t^2 */
    float cy = clamp(q.y, -20.0, -7.0);
    float ct = sqrt((cy + 20.0) / 13.0);
    float cxb = 32.0 * ct - 17.0 * ct * ct;
    float ck = (32.0 - 34.0 * ct) / max(26.0 * ct, 1e-3);
    float cap = min((cxb - ax) / sqrt(1.0 + ck * ck), -7.0 - q.y);
}
```

`ck` is the cap's slope dx/dy : the derivative of x , which is $32 - 34t$, divided by the derivative of y , which is $26t$. The `max(..., 1e-3)` guards the division at the apex, where t is zero. The final `min` with `-7 - q.y` closes the cap with its flat base.

The second half: the nut and the join

The second half solves the nut and joins the two shapes:

```
/* nut: right side (0,18) C (9,18) (15,7) (13,-7); y falls as t rises */
float ny = clamp(q.y, -7.0, 18.0);
float lo = 0.0, hi = 1.0;
for (int i = 0; i < 18; i++) {
    float tt = 0.5 * (lo + hi), uu = 1.0 - tt;
    float by = uu * uu * uu * 18.0 + 3.0 * uu * uu * tt * 18.0
        + 3.0 * uu * tt * tt * 7.0 - tt * tt * tt * 7.0;
    if (by > ny) { lo = tt; } else { hi = tt; }
}
float t = 0.5 * (lo + hi), u = 1.0 - t;
float nxb = 3.0 * u * u * t * 9.0 + 3.0 * u * t * t * 15.0 + t * t * t * 13.0;
float ndx = 3.0 * (u * u * 9.0 + 2.0 * u * t * 6.0 - t * t * 2.0);
float ndy = 3.0 * (2.0 * u * t * -11.0 - t * t * 14.0);
float nk = ndx / min(ndy, -1e-3);
float nut = min(min((nxb - ax) / sqrt(1.0 + nk * nk), 18.0 - q.y), q.y + 7.0);

return min(max(cap, nut) + grow, 17.0 - abs(q.y));
}
```

Eighteen halvings pin `t` to within 2^{-18} of the parameter range, about four millionths. `ndx` and `ndy` are the cubic's derivatives, written out from the control points' differences: 9, 6, -2 across and 0, -11, -14 down. Because `y` always falls along the nut, `ndy` is never positive, and `min(ndy, -1e-3)` keeps the division away from zero at `t = 0`, where the curve runs level across the bottom of the nut.

Union, grow and clip

The last line of the function combines the pieces with three operations:

Operation	Code	Why
union	<code>max(cap, nut)</code>	with positive-inside distances, a point is inside the union if it is inside either shape
grow	<code>+ 0.8</code>	the SVG draws the outline with a stroke; growing by its half-width gives the drawn silhouette
clip	<code>min(..., 17 - abs(q.y))</code>	the 34-unit view box cuts the mark, which is why the watermark had a flat top and bottom

The clip explains two of the curve constants. The cap's apex is at $y = -20$ and the nut's base at $y = 18$, both outside the view box's ± 17 . The box cuts them off, just as it did when the SVG was rendered into the sprite.

The early exit

Nothing of the mark reaches more than 16 units to either side of the centre, or more than 17 above or below it. The cap is the widest part: it is widest where $32 - 34t = 0$, at $x \approx 15.06$, and the 0.8 growth brings that to 15.86. A pixel more than one unit outside that box is plainly outside the mark, so the function returns its distance to the box without solving either curve.

The exit cannot change the picture. Coverage is zero for any pixel more than half a window pixel outside the edge, and the early answer is at least one unit, which is 6.47 guest pixels. The comment in the Metal source states the bound: coverage is zero "for any window taller than 50 pixels, as the exact distance would give". By simple arithmetic, the box plus its one-unit margin is about 12% of the 800×534 background, so the curve solve is skipped for most of the window.

What the exit bought

Commit `e80479db19` rendered both acorn scenes offscreen at 160×120, 400×300, 1646×1156 and 3840×2160. Every output was pixel-identical to the version without the exit. The GPU time per frame changed like this:

Scene	3840×2160, before	3840×2160, after
acorn	0.45 ms	0.095 ms
acorn-live	0.44 ms	0.14 ms

Coverage and colour

The distance is in SVG units. Multiplying by `unitPx × k.y` converts it to window pixels, and adding 0.5 centres a one-pixel ramp on the edge:

Distance from the edge	cover	Colour
more than half a pixel outside	0	the ground, #B7C0B4
on the edge	0.5	halfway between ground and ghost
more than half a pixel inside	1	the ghost, #ADB7A8

The two colours are the ones the old sprite used: the sage ground, and the single ghost tone just off it. The difference between them is only 10, 9 and 12 levels in red, green and blue, which is why the mark reads as a watermark.

acorn-live: two lights and a vignette

The moving scene draws the same acorn and then adds three things: two soft lights that drift slowly over the ground, a faint vignette, and a dither against banding.

```
if (P.bd.w == 2) {
    float tau = 6.2831853 * float(P.bd.y) / 1800000.0;
    float2 uv = px / outPx;
    float aspect = outPx.x / outPx.y;
    float2 l1 = float2(0.5 + 0.34 * sin(7.0 * tau), 0.42 + 0.22 * sin(5.0 * tau + 1.3));
    float2 l2 = float2(0.5 + 0.34 * sin(11.0 * tau + 2.1), 0.58 + 0.22 * cos(9.0 * tau));
    float2 d1 = (uv - l1) * float2(aspect, 1.0);
    float2 d2 = (uv - l2) * float2(aspect, 1.0);
    c += 0.075 * exp(-dot(d1, d1) * 3.5) * float3(1.0, 1.0, 0.96);
    c += 0.05 * exp(-dot(d2, d2) * 4.5) * float3(0.95, 0.72, 0.42);
    float2 v = (uv - 0.5) * float2(aspect, 1.0);
    c *= 1.0 - 0.07 * dot(v, v);
    float n = frac(52.9829189 * frac(dot(px, float2(0.06711056, 0.00583715))));
    c += (n - 0.5) / 255.0;
}
```

The excerpt leaves out the source's comments, which say the same as the table below. On the Mac the test is `BACKDROP == 2`, against the function constant.

What each part does

Part	How	Effect
the clock	<code>bd.y</code> is milliseconds modulo 1,800,000; <code>tau</code> turns it into an angle	one 30-minute loop
light 1	a Gaussian spot, strength 0.075, near-white	drifts at 7 and 5 cycles per loop
light 2	a smaller, weaker spot, strength 0.05, warm orange	drifts at 11 and 9 cycles per loop

What each part does, continued

Part	How	Effect
aspect	offsets scaled by width over height	round lights in a wide window
vignette	darkens with the square of the distance from the centre, about 7% at the corners of a wide window	a faint frame
dither	interleaved gradient noise, less than half a level either way	hides 8-bit banding

The half-hour loop keeps the numbers small. The clock is sent to the GPU as whole milliseconds and converted to a 32-bit float there. 1,800,000 is well below 2^{24} , the range in which a 32-bit float still represents every integer, so the angle loses no precision after a long session. The source comment says as much: the loop is there "so the float time never loses precision".

The dither is needed because the lights and the vignette change the colour very gradually. In 8-bit colour, a gradient that gentle shows visible bands. Adding noise smaller than one level breaks the bands up without visibly speckling the ground.

6. Tiles and pictures

Besides the two acorn scenes, the layer can show an image file in either of two ways: repeated as a tile, or scaled to fill the window as a picture. This chapter covers the value that names a scene, how each host loads an image and what it does with transparency and colour, how big a tile is drawn on each host, how a picture is fitted, and what a picture costs compared with drawing it inside RISC OS.

Naming a scene

The same string names a scene everywhere: in `-display metal,backdrop=` or `-display dx11,backdrop=`, in the Mac's saved `backdrop` setting, and in the value each Backdrop menu item stores:

Value	Scene
off	the feature gated out, which is only possible at start-up (chapter 7)
none	the feature on, nothing drawn: the guest's own sage shows
acorn, acorn-live	the distance-field acorn, still or moving (chapter 5)

Naming a scene, continued

Value	Scene
tile:<file>	the image repeated from the top left of the window
picture:<file>	the image scaled to fill the window, centred, with the overhang cropped
<file>	a bare path is a picture

Parsing the value

Both front ends parse the value in a function called `backdrop_set_scene`. This is the Windows one:

```
if (!spec || !*spec || !strcmp(spec, "none") || !strcmp(spec, "off")) {
    mode = DX11_BACKDROP_OFF;
    spec = "none";
} else if (!strcmp(spec, "acorn")) {
    mode = DX11_BACKDROP_ACORN;
} else if (!strcmp(spec, "acorn-live")) {
    mode = DX11_BACKDROP_ACORN_LIVE;
} else if (!strncmp(spec, "tile:", 5)) {
    mode = DX11_BACKDROP_TILE;
    path = spec + 5;
} else if (!strncmp(spec, "picture:", 8)) {
    mode = DX11_BACKDROP_PICTURE;
    path = spec + 8;
} else {
    mode = DX11_BACKDROP_PICTURE;
    path = spec;
}
if (path) {
    if (backdrop_load_image(path)) {
        if (mode == DX11_BACKDROP_TILE && strstr(path, "@2x")) {
            backdrop.image_scale = 2.0;
        }
    } else {
        mode = DX11_BACKDROP_ACORN;
        spec = "acorn";
        dx11_log("backdrop: cannot read %s; using acorn", path);
    }
}
```

Two behaviours are visible here. First, `off` reaching this function is treated as `none`, because by the time a scene is being set the gate has already been decided. Second, an image that cannot be read falls back to the acorn. The function's comment gives the reason: "so the window is never a mystery"

black". A tile whose path contains @2x is drawn at twice the density, which is covered below.

Loading an image, two ways

Each front end loads images with its platform's own library and converts them to a texture the scale pass can sample. The results are meant to match, but the details differ.

The two loaders compared

	Mac: ImageIO and Core Graphics	Windows: WIC
reads	anything ImageIO can decode, including HEIC	anything WIC has a decoder installed for
transparency	drawn over an opaque sage fill	converted to straight BGRA, then blended over sage in a loop
texture	RGBA8Unorm, no mipmaps	B8G8R8A8_UNORM, immutable, no mipmaps

Sage under the transparent parts

A PNG with transparent areas must not show black where it is transparent, because the texture is later drawn as opaque colour. Both loaders put sage underneath. On the Mac, Core Graphics does it by filling the bitmap context before drawing the image into it:

```
CGContextSetRGBFillColor(ctx, 0xB7 / 255.0, 0xC0 / 255.0,
                        0xB4 / 255.0, 1.0);
CGContextFillRect(ctx, CGRectMake(0, 0, w, h));
CGContextSetInterpolationQuality(ctx, kCGInterpolationHigh);
CGContextDrawImage(ctx, CGRectMake(0, 0, w, h), img);
```

The bitmap context is created with the sRGB colour space, so Core Graphics also converts an image tagged with another colour space into sRGB as it draws.

The Windows blend

On Windows, WIC converts the image to straight (non-premultiplied) BGRA, and a loop does the blend in integers:

```
/* Over the key colour, so transparency lands on sage. */
for (size_t i = 0, n = (size_t)w * h; i < n; i++) {
    uint8_t *q = pix + i * 4;
    unsigned a = q[3];
    q[0] = (uint8_t)((q[0] * a + 0xB4 * (255 - a)) / 255);
    q[1] = (uint8_t)((q[1] * a + 0xC0 * (255 - a)) / 255);
    q[2] = (uint8_t)((q[2] * a + 0xB7 * (255 - a)) / 255);
    q[3] = 0xff;
}
```

The byte order is blue, green, red, alpha, so blue is blended with `0xB4` and red with `0xB7`. For example, a half-transparent pure red pixel (alpha 128) gets a red channel of $(255 \times 128 + 183 \times 127) / 255 = 219$, a little lighter than full red. After the loop every pixel is opaque.

Tiles: an image pixel per point

A tile is sampled with the window pixel's own position, divided by the size of one tile in window pixels:

```
float2 uv = v.pos.xy / (isz * float(P.bx.x) / 1000.0);
bg = bgimg.Sample(rep, uv).rgb;
```

`isz` is the image size and `bx.x` is window pixels per image pixel, times 1000. The `rep` sampler wraps, so the image repeats and each seam is filtered like any other edge. The tile is anchored to the window's top left, not to the guest's desktop, so RISC OS windows move over a pattern that stays still.

The two hosts compute `bx` differently. The Mac works in points, so a plain tile has the same physical size on Retina and standard displays. Windows works in device pixels, with no display scaling factor. The two lines come from different files, so the comments naming them are added here:

```
p->bx[0] = (uint32_t)(px_per_point / backdrop.image_scale * 1000.0 + 0.5); /* Mac */
```

```
p->bx[0] = (uint32_t)(1000.0 / backdrop.image_scale + 0.5);          /* Windows */
```

How big a tile's pixel is drawn

Host and file	Pixels per point	image_scale	bx.x	Window pixels per image pixel
Mac, Retina, plain tile	2	1	2000	2, which is one point
Mac, Retina, @2x tile	2	2	1000	1, one device pixel
Mac, standard display, plain tile	1	1	1000	1
Mac, standard display, @2x tile	1	2	500	0.5
Windows, plain tile	—	1	1000	1
Windows, @2x tile	—	2	500	0.5

Pictures: fill and crop

A picture is scaled by the larger of the two ratios between window and image, so it covers the window in both directions, and then centred:

```
float s = max(outPx.x / isz.x, outPx.y / isz.y);
float2 uv = (v.pos.xy - 0.5 * outPx) / (s * isz) + 0.5;
bg = bgimg.Sample(lin, uv).rgb;
```

In the other direction the picture overhangs the window and is cropped equally at both ends.

A worked fit

The 6016×6016 HEIC picture from MACOS.md, on the Mac, in a 1646×1156 window:

Step	Value
loaded size, after the 4096 cap	4096 × 4096
ratios, width and height; s is the larger	1646 / 4096 = 0.402 , 1156 / 4096 = 0.282
drawn size	1646 × 1646 window pixels
cropped	245 window pixels at the top and at the bottom

What a picture costs

The reason for drawing pictures on the host was cost. MACOS.md measured each way of showing a picture behind the desktop, per full redraw of the 800×534 background by RISC OS, on an M4 Max with GVFill on:

How the picture is drawn	RISC OS, per full redraw	Host GPU, per frame at 1646×1156
inside RISC OS, as a JPEG	48.3 ms	—
inside RISC OS, as a cached sprite	0.505 ms	—
on the host, behind the tagged tile	0.156 ms	0.140 ms

MACOS.md sums it up: against the plain Acorn look the layer is neutral, and against a picture drawn inside RISC OS it is three times cheaper than a sprite and three hundred times cheaper than a JPEG, whatever the host shows.

7. The gate and the menu

A feature like this should cost nothing for someone who does not use it, and be easy to change for someone who does. This chapter covers both sides. First, the gate: the option that decides at start-up whether the layer exists at all. Then the Backdrop menu on each host, the launchers that choose the first scene, and the hint the menu shows when RISC OS is not painting the tag.

Off unless asked for

Since 84dfd3c071, the `backdrop=` display option is both the feature's gate and its first scene. The QAPI schema adds it as an optional string, `'*backdrop' : 'str'`, to both `DisplayMetal` and `DisplayDx11`. If the option is absent or `off`, the feature is left out entirely:

- the decode pass ignores the transfer byte, so tagged pixels show as plain sage;
- nothing is drawn beneath the desktop;
- there is no Backdrop menu.

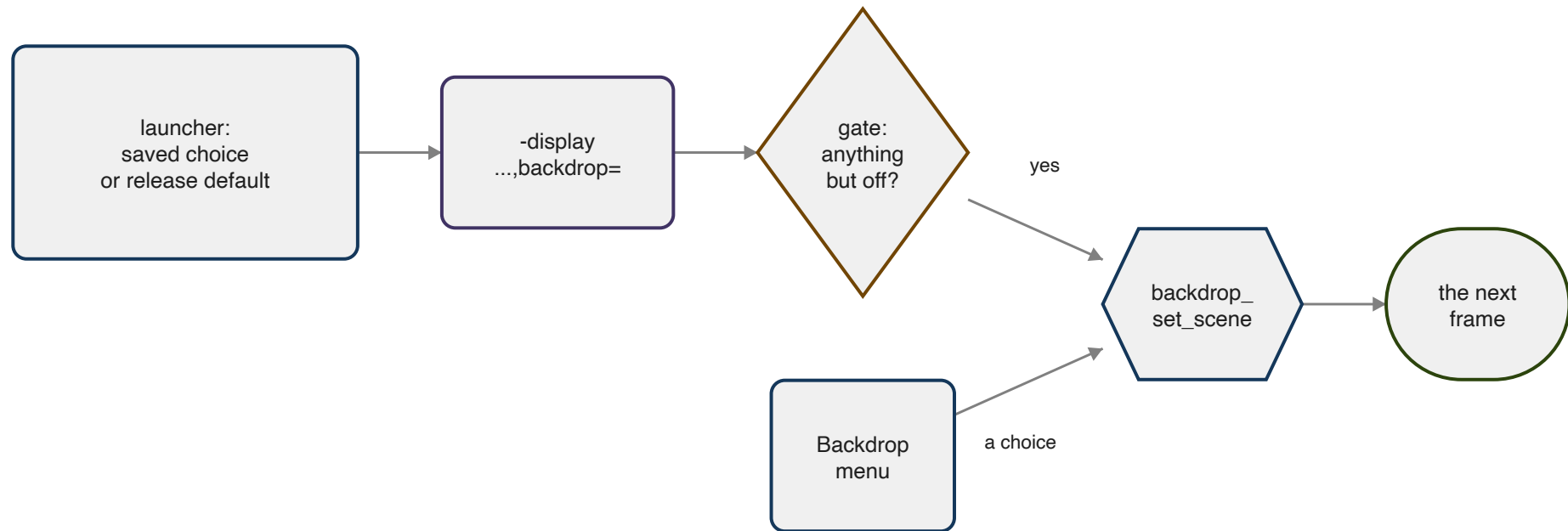
Any other value switches the feature on and names the scene. The gate is decided once, at start-up. After that, the menu only changes the scene.

off and none are different

	off	none
the Backdrop menu	not shown	shown
the decode pass	ignores the byte	ignores the byte, because no scene is drawn
what the window shows	the guest's own pixels	the guest's own pixels
can the menu change it?	no: the gate is closed for the session	yes: pick any scene

The window looks the same either way. The difference is whether the feature is there to switch on. Before 84dfd3c071 the menu's None item meant off, and after it None means "on, with no scene".

How a scene gets chosen



The gate on each host

On the Mac the Backdrop menu item is built early, before the display options are read, so it is created hidden. `ui/metal.c` calls the gate only when the option was given, and the gate shows the item:

```
void metal_glue_backdrop(const char *spec)
{
    backdrop.enabled = spec && *spec && strcmp(spec, "off") != 0;
    [backdrop.menu_item setHidden:!backdrop.enabled];
    if (backdrop.enabled) {
        backdrop_set_scene(spec);
    } else {
        backdrop_set_scene("none");
        metal_log("backdrop: gated off");
    }
}
```

The Windows gate

On Windows the gate builds the submenu itself. The window is created before the display options are read, and the comment on `backdrop_attach_menu` explains the timing: this "is the first moment we know the feature is wanted at all".

```
extern "C" void dx11_glue_backdrop(const char *spec)
{
    backdrop.t0 = GetTickCount64();
    if (!spec || !*spec || !strcmp(spec, "off")) {
        backdrop.enabled = false;
        backdrop.mode = DX11_BACKDROP_OFF;
        snprintf(backdrop.spec, sizeof(backdrop.spec), "off");
        return;
    }
    backdrop.enabled = true;
    backdrop_set_scene(spec);
    backdrop_attach_menu();
    dx11_log("backdrop: %s", backdrop.spec);
}
```

`84dfd3c071` records the Mac check. With no option, and with `backdrop=off`, the Machine menu read Grab Pointer, Save Screenshot, Load Snapshot, Toggle Full Screen, and the log said "gated off". With `backdrop=none`, the Backdrop item was there.

The Mac: Machine › Backdrop

The Mac menu is rebuilt each time it opens, in the menu delegate's `menuNeedsUpdate:`. Rebuilding means that an image dropped into the folder is listed next time, without a restart. This is the middle of that method:

```
[menu addItem:backdrop_item(@"Acorn", @"acorn", self)];
[menu addItem:backdrop_item(@"Acorn, Moving", @"acorn-live", self)];
[menu addItem:backdrop_item(@"None", @"none", self)];
[menu addItem:[NSMenuItem separatorItem]];

sub = [[[NSMenuItem alloc] initWithTitle:@"Tiles" action:NULL keyEquivalent:@""] autorelease];
[sub setSubmenu:backdrop_list(@"Tiles", @"tile",
    @[ backdrop_images([dir stringByAppendingPathComponent:@"Tiles"], ext) ], self)];
[menu addItem:sub];
sub = [[[NSMenuItem alloc] initWithTitle:@"Pictures" action:NULL keyEquivalent:@""] autorelease];
[sub setSubmenu:backdrop_list(@"Pictures", @"picture",
    @[ backdrop_images([dir stringByAppendingPathComponent:@"Pictures"], ext),
      backdrop_images(@"/System/Library/Desktop Pictures", [NSSet setWithObject:@"heic"]) ],
    self)];
[menu addItem:sub];
```

Each item stores its scene string, `tile:` or `picture:` followed by the file's full path, and is ticked when that string matches the scene in force. The Pictures submenu has two groups separated by a line: the user's own pictures, then macOS's own still wallpapers, which are HEIC files. After the submenus come two commands. Open Backdrops Folder creates the folder and its `Tiles` and `Pictures` subfolders if needed, then opens it in the Finder. Choose Backdrops Folder... opens a sheet to pick a different folder.

`backdrop_images` lists a folder in the Finder's own order, using `localizedStandardCompare:`. It skips hidden files, keeps only image extensions (PNG, JPEG, HEIC, HEIF, TIFF, GIF, BMP and WebP), and stops at 200 images.

Saving the choice

A choice applies at once and is saved:

```
- (void)backdropAction:(NSMenuItem *)sender
{
    NSString *spec = [sender representedObject];

    if (spec && backdrop.enabled) {
        backdrop_set_scene([spec UTF8String]);
        [[NSUserDefaults standardUserDefaults] setObject:backdrop.spec forKey:@"backdrop"];
    }
}
```

Three settings on the Mac

Setting	Where it lives	Written by	Read by
backdrop	the app's user defaults	the menu, or defaults write	the launcher, at the next start
backdropFolder	the app's user defaults	Choose Backdrops Folder...	the menu, each time it opens
RISCOSBackdrop	the app's Info.plist	make-release.sh	the launcher, when backdrop is not set

The launcher's order

The launcher, `app/launcher.zsh`, resolves the scene in that order and passes it on:

```
BACKDROP="$(pref backdrop)"
[[ -n "$BACKDROP" ]] || BACKDROP="$(/usr/libexec/PlistBuddy -c 'Print :RISCOSBackdrop' "$CONTENTS/Info.plist" 2>/dev/null)"
BACKDROP="${BACKDROP:-off}"
```

It then starts QEMU with `-display "metal,vsync=30,backdrop=$(q "$BACKDROP")"`. The last fallback is `off`, so an app built without the setting starts without the feature. A user's saved choice always beats the release's default.

Windows: the window menu

The Windows front end adds a Backdrop submenu to the window menu, the menu behind the title bar's icon. Items in that menu report through `WM_SYSCOMMAND`, and the system reserves the low four bits of the command, so every identifier is a multiple of 16:

```
#define DX11_SC_BD_ACORN 0x0200
#define DX11_SC_BD_LIVE 0x0210
#define DX11_SC_BD_NONE 0x0220
#define DX11_SC_BD_OPEN 0x0230
#define DX11_SC_BD_IMG0 0x0300 /* one per listed image, step 16 */
#define DX11_SC_BD_IMGS 200 /* as many as the Metal menu lists */
```

From an identifier back to a scene

Images take identifiers from 0x0300 upwards, 16 apart. A table of 200 strings, backdrop_menu_spec, holds the scene each one selects. The window procedure rebuilds the submenu on WM_INITMENUPOPUP whenever it is about to open, and passes each command to backdrop_command, which turns an identifier back into a slot:

```
if (id >= DX11_SC_BD_IMG0 && id < DX11_SC_BD_IMG0 + DX11_SC_BD_IMGS * 16
    && (id - DX11_SC_BD_IMG0) % 16 == 0) {
    char *spec = backdrop_menu_spec[(id - DX11_SC_BD_IMG0) / 16];

    if (spec) {
        backdrop_set_scene(spec);
        return true;
    }
}
return false;
```

The window procedure checks its snapshot and HostNet items first. When backdrop_command returns false as well, the command goes to Windows' default handling.

The Windows menu, against the Mac's

	Mac	Windows
where	Machine > Backdrop	the window menu, Backdrop
folder	backdropFolder, or ~/Pictures/RISC OS Backdrops	QEMU_BACKDROPS, or %USERPROFILE%\Pictures\RISC OS Backdrops
order	the Finder's, localizedStandardCompare:	Explorer's: CompareStringEx with digits as numbers, case ignored

The hint: is RISC OS painting the tag?

A backdrop can only show through a disc that tiles the tagged sprite, in a 32bpp mode. When either is missing, choosing a scene would appear to do nothing. So both menus open with a note when the guest is not painting the tag. The Windows version of the check:

```
static int backdrop_guest_state(void)
{
    Dx11FbView v;

    if (!fb.up || !dx11_glue_fb_view(&v) || !v.fb) {
        return -2;
    }
    if (v.bpp != 32) {
        return -1;
    }
    for (uint32_t y = 0; y < v.yres && y + v.yoffset < v.rows; y += 7) {
        const uint8_t *row = (const uint8_t *)v.fb
            + (size_t)(y + v.yoffset) * v.pitch;

        for (uint32_t x = 0; x < v.xres; x += 7) {
            const uint8_t *px = row + (size_t)(x + v.xoffset) * 4;
            uint32_t rgb = v.pixo ? (px[0] | px[1] << 8 | px[2] << 16)
                : (px[2] | px[1] << 8 | px[0] << 16);

            if ((px[3] & 0xC0) == 0x80 && rgb == DX11_BACKDROP_KEY) {
                return 1;
            }
        }
    }
    return 0;
}
```

What the menu shows

The function samples one pixel in every 7 across and 7 down, 1 in 49, and stops at the first tagged key pixel. The Mac version samples the newest frame uploaded to the GPU instead of the live framebuffer, with the same grid and results:

Result	Meaning	The note at the top of the menu
1	a tagged key pixel was found	none
0	a 32bpp mode, but no tag found	"RISC OS is not drawing a backdrop to show through"
-1	not a 32bpp mode	"Backdrops need a 16 million colour screen mode"
-2	no frame yet	none

`1cc6d04a18` records the check: the hint appeared when the guest switched back to its untagged watermark.

The launchers and the test harness

The Windows launcher is a small C program compiled by `make-release.py`. The scene is a compile-time constant, defaulting to the acorn:

```
#ifndef BACKDROP
#define BACKDROP L"acorn"
#endif
```

It is spliced into the emulator's command line as `L"-display dx11,backdrop=" BACKDROP L" -serial null "`. Before `d535505fad`, the launcher passed a bare `-display dx11`, and the commit message records the result: the installed Windows app had no Backdrop menu at all.

For development, `run.py --backdrop SCENE` adds the option to a Windows run. As on the command line, the feature is off unless the option is given.

8. Capturing what is on screen

Once the host draws part of the picture itself, a screen capture has to decide which picture it means. This chapter covers the change `c4024d236f` made on Windows, where PrintScreen and QMP `screendump` now save what the window shows. That involves a copy taken between the last draw and the flip, a handshake between two threads, and a small hook in QEMU. The chapter then covers the Mac's screenshot, which renders the composite again, and the test harness change that followed from all of this.

Two pictures of one screen

With the layer on, the guest's framebuffer and the window no longer hold the same picture. Where the window shows the acorn, the framebuffer holds plain sage with a tag. The window is also scaled to its own size, and the pointer is drawn over it by the host. Before `c4024d236f`, both Windows captures saved the decoded framebuffer. The commit message describes that as "what the guest drew, at the mode's own size, with no backdrop and no pointer".

What each capture saves now

Capture	Saves	Size	Backdrop	Pointer
Windows, PrintScreen	the composited frame	the window's	yes	yes
Windows, screendump	the composited frame	the window's	yes	yes
Windows, screendump with a device	the guest's framebuffer	the mode's	no	no
Mac, Save Screenshot	the composite, drawn again	the mode's	yes	yes
Mac, screendump	the guest's framebuffer	the mode's	no	no

MACOS.md records the Mac's behaviour: screenshots hold the composite, and QMP `screendump` still returns the guest's own pixels. The hook that changes `screendump` is only registered by the Direct3D front end.

Windows: copy the back buffer before the flip

The composited picture only ever exists in the swap chain's back buffer. The window uses a flip-model swap chain, and in the flip model the back buffer's contents are undefined after `Present`. The copy therefore has to be taken inside the frame, after the pointer is drawn and before the flip. The frame loop does exactly that:

```
if (dx11_acquire_target()) {
    if (!dx11_render_frame()) {
        dx11.context->ClearRenderTargetView(dx11.rtv, clear);
    }
    /* The screen as it will appear, while the back buffer still
     * holds it: the flip leaves its contents undefined. */
    comp_capture();
    HRESULT hr = dx11.swap->Present(1 /* vsync */, 0);
}
```

The capture's state

`comp_capture` returns at once unless someone has asked for a copy, so an ordinary frame pays only for the test. Its state lives in one structure:

```
static struct {
    CRITICAL_SECTION lock;      /* over the buffer; held while a reader has it */
    CRITICAL_SECTION turn;      /* one reader at a time */
    HANDLE ready;               /* set when a copy has landed */
    ID3D11Texture2D *staging;
    uint8_t *pixels;            /* BGRA, w * h * 4 */
    uint32_t w, h;
    size_t cap;
    bool want;                  /* a copy has been asked for */
    bool png;                   /* ... and PrintScreen wants a file of it */
    bool up;
} comp;
```

Copying the frame out

When a copy is wanted, `comp_capture` gets the back buffer and copies it to a *staging* texture, which is the kind the CPU is allowed to map. The staging texture is kept between captures and rebuilt only when the window's size or format changes. The rest of the function maps it and copies the rows out:

```
dx11.context->CopyResource(comp.staging, back);
back->Release();

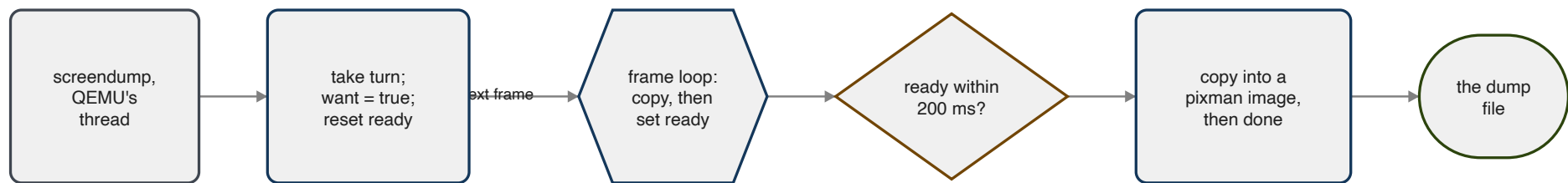
if (FAILED(dx11.context->Map(comp.staging, 0, D3D11_MAP_READ, 0, &map))) {
    return;
}
EnterCriticalSection(&comp.lock);
png = comp.png;
comp.png = false;
comp.want = false;
if (comp.cap < (size_t)bd.Width * bd.Height * 4) {
    free(comp.pixels);
    comp.cap = (size_t)bd.Width * bd.Height * 4;
    comp.pixels = (uint8_t *)malloc(comp.cap);
}
if (comp.pixels) {
    for (uint32_t y = 0; y < bd.Height; y++) {
        memcpy(comp.pixels + (size_t)y * bd.Width * 4,
               (const uint8_t *)map.pData + (size_t)y * map.RowPitch,
               (size_t)bd.Width * 4);
    }
    comp.w = bd.Width;
    comp.h = bd.Height;
}
LeaveCriticalSection(&comp.lock);
SetEvent(comp.ready);
```

The rows are copied one at a time because a mapped texture's row pitch can be wider than four bytes per pixel. The buffer grows when the window does and is never shrunk. If PrintScreen asked, the PNG is written from the still-mapped data after the lock is released, and then the texture is unmapped.

The handshake with QEMU's thread

A screendump arrives on QEMU's main loop thread, while the frame loop runs on the window's own thread. The two meet through the two critical sections and a manual-reset event:

One screendump, step by step



The reader's half

The reader's half is `dx11_glue_composite_get`:

```
extern "C" int dx11_glue_composite_get(uint32_t *w, uint32_t *h,
                                       const void **pixels, int wait_ms)
{
    ...
    EnterCriticalSection(&comp.turn);
    EnterCriticalSection(&comp.lock);
    ResetEvent(comp.ready);
    comp.want = true;
    LeaveCriticalSection(&comp.lock);

    WaitForSingleObject(comp.ready, wait_ms < 0 ? 0 : (DWORD)wait_ms);

    EnterCriticalSection(&comp.lock);
    if (!comp.pixels || !comp.w || !comp.h) {
        LeaveCriticalSection(&comp.lock);
        LeaveCriticalSection(&comp.turn);
        return 0;          /* no frame yet: the caller falls back */
    }
    *w = comp.w;
    *h = comp.h;
    *pixels = comp.pixels;
    return 1;
}
```

It returns holding both critical sections. `dx11_glue_composite_done` releases them. While a reader holds `lock`, the frame loop cannot overwrite the pixels being read, because `comp_capture` must take the same lock. `turn` keeps a second reader from interleaving with the first.

The QEMU side: a hook for compositing front ends

QEMU's `screendump` did not know that a front end could hold a better picture than the console's surface. `c4024d236f` adds a hook to `include/ui/console.h`:

```
typedef pixman_image_t *(*QemuUiComposite)(void);
void qemu_ui_set_composite(QemuUiComposite fn);
pixman_image_t *qemu_ui_composite(void);
```

screendump asks the hook first

`ui/console.c` keeps the function in a single static pointer, and `qemu_ui_composite` calls it if it is set. In `ui/ui-qmp-cmds.c`, `qmp_screendump` asks the hook first, unless the command named a device:

```
image = device ? NULL : qemu_ui_composite();
if (!image) {
    /* No compositing front end, or it has not drawn a frame yet:
     * the guest's own framebuffer, as before. */
    surface = qemu_console_surface(con);
    if (!surface) {
        error_setg(errp, "no surface");
        object_unref(con);
        return;
    }
    image = pixman_image_ref(surface->image);
}
```

Naming a device is therefore how to get the guest's own framebuffer back on Windows. When no front end has registered, the hook returns `NULL` and the command behaves exactly as before.

The Direct3D hook

The Direct3D front end registers the hook in `dx11_display_init`. Its function wraps the handshake:

```
static pixman_image_t *dx11_composite_image(void)
{
    const void *pixels;
    uint32_t w = 0, h = 0;
    pixman_image_t *image;

    if (!dx11_glue_composite_get(&w, &h, &pixels, 200)) {
        return NULL;
    }
    image = pixman_image_create_bits(PIXMAN_x8r8g8b8, w, h, NULL, 0);
    if (image) {
        memcpy(pixman_image_get_data(image), pixels, (size_t)w * h * 4);
    }
    dx11_glue_composite_done();
    return image;
}
```

Blue, green, red and alpha bytes in memory are pixman's `x8r8g8b8` format on a little-endian host. The "x" means the fourth byte is ignored, and the source comment explains why: "the alpha the compositor leaves behind is not ours to interpret".

PrintScreen writes a PNG

PrintScreen sets both `want` and `png`, so the frame loop writes the file itself. `comp_write_png` is a minimal PNG writer. It emits the signature and an IHDR for 8-bit RGB, then turns each BGRA row into a filter byte of 0 followed by RGB bytes. The whole image is compressed with zlib's `compress2` at level 6 into one IDAT chunk, and an IEND closes the file. The file is named `dx11-screenshot-001.png`, then `002` and so on, in the emulator's current directory, and the numbering starts again at each run.

The Mac: render the composite again

The Mac's Save Screenshot does not copy the window. It reads the decoded texture back at the guest's resolution. With a backdrop, the decoded texture has holes in it, so the screenshot first runs the scale pass once more, at 1:1, into an RGBA8 texture:

```
if (backdrop.mode != METAL_BACKDROP_OFF) {
    if (!fb.shot) {
        MTLFunctionConstantValues *scv = [[MTLFunctionConstantValues alloc] init];
        uint32_t nearest = METAL_SCALING_NEAREST;
        bool no = false;
        uint32_t bd = (uint32_t)backdrop.mode;

        [scv setConstantValue:&nearest type:MTLDataTypeUInt atIndex:1];
        [scv setConstantValue:&no type:MTLDataTypeBool atIndex:2];
        [scv setConstantValue:&bd type:MTLDataTypeUInt atIndex:3];
        fb.shot = metal_pipeline(@"ps_scale", scv, MTLPixelFormatRGBA8Unorm);
        [scv release];
    }
}
```

The screenshot pipeline is the window's scale pipeline specialised differently: nearest scaling, since the output is 1:1, and no scanlines. It is cached in `fb.shot`, and `fb_release_scale()` drops it together with the window's pipeline when the scene changes (chapter 4).

A tile must look the same size relative to the desktop in the screenshot as in the window, so its scale is carried into guest pixels:

```
/* the window's tile size, carried into guest pixels */
backdrop_params(&sp, m.layer.contentsScale * (double)fb.yres
               / MAX(m.layer.drawableSize.height, 1.0));
```

A worked tile size

An 800×600 mode in a Retina window 1156 device pixels tall, with a plain tile:

Quantity	In the window	In the screenshot
pixels per point passed to backdrop_params	2	$2 \times 600 / 1156 = 1.038$
output pixels per tile pixel	2 device pixels	1.038 guest pixels
output pixels per guest pixel	$1156 / 600 = 1.927$ device pixels	1
tile pixel ÷ guest pixel	$2 / 1.927 = 1.038$	1.038

The last row matches, which is the point: a tile covers the same share of the desktop in both pictures. After the readback, the pointer is blended into the pixels with the same straight alpha the window uses, and the file is written as `metal-screenshot-N.png` in the app's Application Support folder.

The test harness: a screen that never settles

`riscos-pi4/tools/run.py` starts a Windows machine and waits for the desktop by polling `screendump` twice a second. It used to decide that the desktop had settled when two dumps were byte-for-byte equal for two seconds. Once `screendump` returned the composite, that could fail for ever: the moving scene changes almost every pixel by a level on every frame, while the guest draws nothing. The comparison is now sampled and tolerant. Here it is without its docstring:

```
def frames_alike(a, b, step=97, tol=8, allow=0.005):
    if b is None or len(a) != len(b):
        return False
    n = bad = 0
    for i in range(0, len(a), step):
        n += 1
        if abs(a[i] - b[i]) > tol:
            bad += 1
    return bool(n) and bad <= allow * n
```

It reads one byte in every 97 of the dump's pixel data and counts a sample as changed only if it differs by more than 8 levels. The two dumps count as alike when no more than half a percent of samples changed. The commit message states the calibration: the threshold "still ignores a blinking caret and still catches a line of text".

Two smaller changes came with it. The poll's "is anything drawn yet?" test changed from "any non-zero byte" to "any byte brighter than 40", still among the first 48,000 bytes of pixel data, because a composited dump taken before the first guest frame shows the window's clear colour, (13, 13, 20), rather than black. And `run.py` had declared the `usb-net` device before the USB hub it plugs into, so every launch with the emulated network card stopped with "usb port 1.3 not found". The device is now declared after the hub.

9. Measured, and in progress

This chapter collects the numbers in one place: what the backdrop costs RISC OS, and what each scene costs the GPU. It ends with the work on the backdrop that was in progress, but not committed, on 15 September.

What RISC OS pays

All measurements are from MACOS.md §7a, taken on an M4 Max with the end-user disc and GVFill on. The first table is the cost to RISC OS of one redraw of the whole 800×534 background. Pinboard makes that redraw, in pieces, whenever a window moves over the background:

One full background redraw

Background	RISC OS, per full redraw
the old centred watermark, drawn by RISC OS	0.118 ms
the tagged tile, 256 pixels (the default)	0.156 ms
the tagged tile, 32 pixels	0.170 ms
a picture as a cached sprite, drawn by RISC OS	0.505 ms
a picture as a JPEG, drawn by RISC OS	48.3 ms

The tile costs 0.038 ms more than the old watermark per full redraw, which MACOS.md calls neutral. What the host then draws through the tile, even a large picture, costs RISC OS nothing more. Against a picture drawn inside RISC OS, the layer is about 3 times cheaper than a cached sprite and 300 times cheaper than a JPEG.

What the GPU pays

The second table is the scale pass's GPU time per frame, from an offscreen benchmark of the Metal shader:

Scene	1646×1156 window	3840×2160 window
off	0.034 ms	0.054 ms
acorn	0.037 ms	0.095 ms
acorn-live	0.040 ms	0.136 ms
picture, a 4096×4096 image	0.140 ms	0.122 ms
tile	0.019 ms	0.058 ms

The "off" row is the scale pass with no layer, so the other rows can be read against it. The most expensive result, 0.14 ms, is less than 1% of a 60 Hz frame's 16.7 ms. Before the acorn's early exit (chapter 5), the acorn cost 0.45 ms at 4K.

Work in progress

The local checkout on 15 September held uncommitted changes to `ui/metal.m` and `MACOS.md`, and a new, untracked `tools/mktiles.py`; this describes them as they stood. The Mac's Backdrop menu was becoming a browser for large tile collections: a submenu for each folder, filled only as it opens (`BackdropListMenu`), with a 32×16-point swatch for every tile or picture and names without the `@2x` marker.

`mktiles.py` builds ready-made tile folders from four freely licensed sets, 196 tiles in all: RISC OS's own desktop textures (Apache 2.0), the CDE desktop's backdrops (CC BY-SA 3.0), the X.Org `xbitmaps` patterns (MIT-style) and Kenney's Pattern Pack Pixel (CC0). Each source pixel becomes a 2×2 block in an `@2x` PNG, so it keeps its hard edges, and each folder gets an "About these tiles.txt" with the source, the licence and what was changed or left out.